

Pokémon TCG Pocket

Deck Meta-Analyzer

5 Members · 3-Week Timeline

Context

In Pokémon TCG Pocket, players build 20-card decks from a limited pool acquired via random booster packs. The competitive metagame is well-tracked: tournament organizers like Limitless TCG publish deck lists, placements, head-to-head match results, and usage statistics for every recorded event, while community databases mirror the full card catalog as structured JSON.

The original proposal framed this as a “deck recommender,” which collapses into “build the best deck you can afford” a budgeting tool, not a research project. This revised proposal reframes the work around predicting a deck’s expected performance from the meta ecology and understanding what cards drive that performance. This treats deckbuilding as an empirical question with a measurable answer rather than an optimization with a trivial solution.

Data Sources

Both sources will be ingested by a single `data_ingest.py` module on first run and cached locally so the rest of the pipeline is fully deterministic.

Card Catalog

flibustier/pokemon-tcg-pocket-database on GitHub. Raw JSON of every card across all released sets.

- URL: <https://raw.githubusercontent.com/flibustier/pokemon-tcg-pocket-database/main/dist/cards.json>
- No API key required.
- Basic fields (`cards.json`): set codes, rarity, name, element, HP, attacks, energy costs.
- Detailed fields (`cards.extra.json`): abilities and full attack text, this file must also be ingested for complete card data.
- We will scope to a fixed era (one or two recent sets, frozen at project start) for reproducibility.

Tournament Data

Limitless TCG public API (docs.limitlesstcg.com/developer.html). Provides tournament placements, full deck lists, and match-level results for Pokémon TCG Pocket.

- Base URL: <https://play.limitlesstcg.com/api>
- Most endpoints (placements, matches) do NOT require an API key.

- The /decks endpoint DOES require an API key. Since this endpoint is essential for deck list access, the team should apply for a key at play.limitlesstcg.com/account/settings/api at the start of the project. Keys are only granted to public-facing projects with a legitimate use-case.
- All responses will be cached to local JSON to make downstream work deterministic.

[Basic Goal — 40%]

Basic Goal: Card & Deck list Ingestion + Collection Analysis

What It Does

Automatically pull and parse the card catalog and a curated set of tournament-topping deck lists for the chosen era. Build Card, Deck, and Collection classes using OOP principles. Compute, for a given user collection, the completion percentage and the exact missing cards for each meta deck.

Why It's a Real Basic Goal

The novelty over the original version is that the catalog is now ingested automatically from a named public database, not handwritten. This is the foundation every other goal builds on, and it's directly verifiable: if the parser handles the published JSON schema and the set-overlap math is correct, downstream code can trust it.

Tests & Sanity Checks

- Schema validation: every parsed card has the required fields; total card count matches the set's published size.
- Round-trip serialization: dump and reload a Collection to JSON, confirm equality.
- Hand-crafted overlap cases: an empty collection → 0% on every deck; a collection equal to a deck → 100%; partial collections give the expected card-by-card diff.
- Property-based test: random subsets of a deck yield completion percentages that match a naïve reference implementation.

[Intermediate Goal — 20%]

Intermediate Goal: Expected Win Rate from Meta Ecology

What It Does

Given any deck list, a meta deck, a user-built variant, or a hypothetical, estimate its expected win rate against the current meta field. The estimate uses the matchup matrix and field-share data from Limitless:

$$E[WR] = \sum (d \in \text{meta}) P(\text{opponent plays } d) \times WR(\text{our deck}, d)$$

We identify “our deck” by archetype-matching it against known meta-archetypes using a deck-fingerprint distance over the card list, then look up its row in the empirical matchup matrix. For

off-meta decks or those with no recorded matches, we fall back to a similarity-weighted average over the nearest known archetypes.

Why This Is a Proper Intermediate Goal

It's no longer a static tier list, it's a model that produces a numeric expected score for any deck, including ones that have never been played, by reasoning about field composition. This is the kind of analysis competitive players want.

Tests & Sanity Checks

- Reproduce known win rates: feed in a top-tier meta deck and confirm the predicted expected WR matches Limitless's published WR within tolerance.
- Field invariance: predicted WRs across all decks, weighted by meta share, should sum to ~0.5 (a closed zero-sum system).
- Monotonicity: if we artificially raise the share of a deck our archetype beats, our expected WR must rise.
- Held-out validation: train field weights on tournaments from month N, predict expected WRs for month N+1's top finishers, report correlation.

[Advanced Goal — 20%]

Advanced Goal: Card Role Classification & Win-Rate Attribution

What It Does

Classify every card appearing in tournament deck lists into one of a small, fixed taxonomy of roles — win condition, engine, staple, tech, garnet/filler, using unsupervised features (co-occurrence with other cards, usage rate across archetypes, position in the deck-frequency distribution, energy/trainer category). Then, for each role, fit a regression measuring how that role's quality predicts the deck's observed tournament win rate.

The output is a table answering: which role contributes most to a deck's win rate? Are top decks winning because of their win conditions, their engine consistency, their tech choices, or staples? We can then re-examine the user's collection through this lens, e.g. "You're missing the engine cards for Deck X, which historically explain 35% of its win-rate variance."

Why This Has a Clear Evaluation Strategy

Unlike substituting a card and guessing a new win rate (which produces unseen deck lists that can't be validated), this goal evaluates against observed tournament data. Classification quality is checked by holdout consistency and against published community labels from PokemonMeta, and the role-importance regression is checked by held-out R^2 on tournaments the model didn't see during fitting.

Tests & Sanity Checks

- Classifier sanity: cards in $\geq 80\%$ of all decks for an archetype get tagged staple/engine; cards in $\leq 20\%$ get tech/garnet.

- Stability: rerun the classifier on a bootstrap resample of tournaments; role assignments should agree on >90% of cards.
- Regression diagnostics: report R^2 on a held-out set of tournaments, plus residual plots.
- Counterfactual sanity: when we zero out the engine-role contribution in the regression, predicted win rates collapse toward the mean, confirming the model assigns explanatory weight in the expected direction.

[Visualizations & Readability — 20%]

Visualizations & Readability

- Heatmap of the matchup matrix between top archetypes (intermediate goal).
- Bar chart comparing observed vs. predicted expected WR per deck (intermediate goal validation).
- Stacked bar per archetype showing relative win-rate contribution of each card role (advanced goal).
- Terminal table summarizing, for the user's collection: top 5 decks by completion %, expected WR if fully built, and which role's cards they're missing.

Matplotlib for all charts; tabulate for terminal output. PEP 8 enforced via flake8 in CI; type hints throughout; docstrings on every public class and function.

Plan & Division of Labor

5 members, each contributing approximately 20% of the total project work. Work is structured so Members D and E run in parallel during Week 3 and should coordinate early on shared data formats for role-attribution outputs.

Member	Role	Responsibilities	Share
Member A	Data & Ingestion	data_ingest.py module, card catalog parsing (cards.json + cards.extra.json), Limitless API scraping, local JSON caching, schema validation	~20%
Member B	OOP & Basic Goal	Card, Deck, Collection classes; collection-vs-deck overlap logic; completion % calculator; full basic goal test suite (schema, round-trip, property-based)	~20%
Member C	Intermediate Goal	Archetype fingerprint matching, matchup matrix construction from Limitless, expected WR estimator, held-out validation	~20%
Member D	Advanced Goal	Card-role feature engineering, unsupervised role classifier, role-importance regression, attribution diagnostics	~20%

Member	Role	Responsibilities	Share
Member E	Visualization & Integration	Matchup heatmap, WR bar charts, stacked role contribution bars, terminal table output, PEP 8/flake8 pass, README, integration tests	~20%

3-Week Timeline

Note: The original proposal included a 4-week timeline. This has been revised to 3 weeks by running the advanced goal and visualization work in parallel during Week 3.

Week	Focus	Key Tasks
Week 1	Data Foundation & OOP Scaffolding	- Lock the card era and freeze the dataset scope - Member A: scrape and cache card catalog (cards.json + cards.extra.json) and one month of Limitless tournament data; finalize schemas - Member B: build Card, Deck, Collection OOP classes and overlap/completion logic - Note: the Limitless /decks endpoint requires an API key, apply early and cache responses locally - All: agree on shared data contracts between modules
Week 2	Basic Goal Complete + Intermediate Goal	- Member B: finalize basic goal with full test suite (schema validation, round-trip serialization, property-based tests) - Member C: build archetype fingerprint matcher, construct matchup matrix, implement expected WR estimator - Member D: begin card-role feature engineering and co-occurrence analysis - Member E: prototype terminal table output and start matchup heatmap
Week 3	Advanced Goal, Visualization & Polish	- Member C: complete held-out validation for intermediate goal - Member D: finalize unsupervised role classifier, fit role-importance regression, run diagnostics - Member E: produce all final visualizations; PEP 8/flake8 pass - All: integration testing across the full pipeline; write final report/README - Members D & E: coordinate early on shared data format for role-attribution outputs